Writing sample 1 of 4: Tutorial (task-based, docs-as-code). Prepared by Judah Paemka for the Grafana Labs Staff Technical Writer application, October 2026. Original work written to Grafana Writers' Toolkit conventions and checked with the Grafana Vale style rules; not published Grafana documentation. The Alloy pipeline, the labels output, and every LogQL query were run end to end against Alloy v1.21.0 and Loki 3.7.9.

PAGE FRONT MATTER (HUGO, DOCS-AS-CODE SOURCE)

```
title: Collect and label JSON application logs with Grafana Alloy
menuTitle: Collect JSON application logs
description: Learn how to use Grafana Alloy to read JSON application logs from files, turn a low-cardinality field into a Loki label, send the logs to Loki, and query them in Grafana with LogQL.
weight: 300
keywords:
  - Alloy
  - Loki
  - logs
  - labels
  - LogQL
labels:
  products:
    - oss
```

Collect and label JSON application logs with Grafana Alloy

Many applications write structured JSON logs to local files. Grafana Alloy can read those files, decide which fields become Loki labels, and send the logs to Loki, where you query them with LogQL.

In this tutorial, you run a small sample application that writes JSON order logs. You configure Alloy to collect the logs, turn the `level` field into a label, and send the results to Loki. Then you query the logs in Grafana and learn why some fields belong in labels and others don't.

In this tutorial, you learn how to:

- Find and read log files with the `local.file_match` and `loki.source.file` components
- Parse JSON log lines and create labels with the `loki.process` component
- Send logs to Loki with the `loki.write` component
- Check component health in the Alloy UI
- Query logs, and create a metric from logs, with LogQL

This tutorial takes about 20 minutes.

How the sample environment works

The sample environment runs four containers with Docker Compose:

Container	Role
`log-generator`	Writes one JSON log line per second to `/var/log/orders/orders.log`. Every tenth line is an error.
`alloy`	Reads the log file, parses each line, adds labels, and sends the logs to Loki.
`loki`	Stores the logs and indexes their labels.
`grafana`	Queries Loki so you can explore the logs.

The log file lives in a Docker volume that two containers share. The log generator writes to the volume, and Alloy reads from it.

```
log-generator --> orders.log (shared volume) --> Alloy --> Loki --> Grafana
```

Before you begin

To complete this tutorial, you need:

- Docker Engine or Docker Desktop with Docker Compose v2. To check, run `docker compose version`.
- Ports `3000`, `3100`, and `12345` available on your machine.
- A text editor and a terminal.

You don't need prior experience with Alloy or Loki.

> **NOTE**
> This tutorial uses Grafana Alloy v1.21.0, Loki 3.7.9, and Grafana 13.2.4. If you use other versions, the output you see can differ slightly.

Create the project files

Create a directory for the tutorial and change into it:

```
mkdir alloy-json-logs && cd alloy-json-logs
```

You create four files in this directory.

Create the log generator

Create a file named `generate-logs.sh` with the following content. The script writes one JSON log line per second, and every tenth line is an error.

```
#!/bin/sh
mkdir -p /var/log/orders
i=0
while true; do
  i=$((i + 1))
  level="info"
  msg="order processed"
  if [ $((i % 10)) -eq 0 ]; then
    level="error"
    msg="payment declined"
  fi
  echo "{\"level\":\"$level\",\"msg\":\"$msg\",\"order_id\":$i}" >> /var/log/orders/orders.log
  sleep 1
done
```

Each line the script writes looks like this:

```
{"level":"info","msg":"order processed","order_id":41}
```

Create the Alloy configuration

Create a file named `config.alloy` with the following content:

```
// Find the log files to collect.
local.file_match "orders" {
  path_targets = [{"__path__" = "/var/log/orders/*.log", "job" = "orders-app"}]
}

// Read new lines from each matched file.
loki.source.file "orders" {
  targets    = local.file_match.orders.targets
  forward_to = [loki.process.orders.receiver]
}

// Parse each JSON line and turn the level field into a label.
loki.process "orders" {
  stage.json {
    expressions = {level = ""}
  }

  stage.labels {
    values = {level = ""}
  }

  forward_to = [loki.write.local.receiver]
}

// Send the processed log lines to Loki.
loki.write "local" {
  endpoint {
    url = "http://loki:3100/loki/api/v1/push"
  }
}
```

This configuration is a pipeline of four components. Each component passes its output to the next one, either through an exported value such as `targets` or through the `forward_to` argument.

Component	What it does in this pipeline
`local.file_match "orders"`	Finds files that match `/var/log/orders/*.log` and attaches the label `job="orders-app"` to them.
`loki.source.file "orders"`	Reads each matched file and sends each line it reads to `loki.process`.
`loki.process "orders"`	Parses each line as JSON, extracts the `level` field, and turns it into a label.
`loki.write "local"`	Sends the processed log entries to the Loki push API.

In `stage.json`, `expressions = {level = ""}` extracts the JSON field named `level`. An empty string means the extracted value uses the same name as the key. In `stage.labels`, `values = {level = ""}` creates a label from that extracted value.

Provision the Loki data source

Create a file named `grafana-datasources.yaml`. Grafana reads this file at startup and adds Loki as a data source, so you don't need to configure it in the UI.

```
apiVersion: 1

datasources:
  - name: Loki
    type: loki
    access: proxy
    url: http://loki:3100
    isDefault: true
```

Create the Docker Compose file

Create a file named `docker-compose.yaml` with the following content:

```
services:
  log-generator:
    image: busybox:1.37
    command: ["sh", "/scripts/generate-logs.sh"]
    volumes:
      - ./generate-logs.sh:/scripts/generate-logs.sh:ro
      - orders-logs:/var/log/orders

  loki:
    image: grafana/loki:3.7.9
    ports:
      - "3100:3100"

  alloy:
    image: grafana/alloy:v1.21.0
```

```
    command:
      - run
      - --server.http.listen-addr=0.0.0.0:12345
      - --storage.path=/var/lib/alloy/data
      - /etc/alloy/config.alloy
    volumes:
      - ./config.alloy:/etc/alloy/config.alloy:ro
      - orders-logs:/var/log/orders:ro
    ports:
      - "12345:12345"
    depends_on:
      - loki

  grafana:
    image: grafana/grafana:13.2.4
    environment:
      - GF_AUTH_ANONYMOUS_ENABLED=true
      - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
      - GF_AUTH_DISABLE_LOGIN_FORM=true
    volumes:
      - ./grafana-datasources.yaml:/etc/grafana/provisioning/datasources/datasources.yaml:ro
    ports:
      - "3000:3000"
    depends_on:
      - loki

volumes:
  orders-logs:
```

The `:ro` suffix mounts the shared volume as read-only for Alloy, because Alloy only needs to read the log file.

> **WARNING**
> This Grafana configuration turns on anonymous access with the Admin role so you can skip the sign-in step. Use it only on your local machine.

Start the environment

1. Start the containers in the background:

   ```
   docker compose up -d
   ```

2. Check that all four containers are running:

   ```
   docker compose ps
   ```

   The `STATUS` column shows `Up` for each container.

3. Check that Loki is ready:

   ```
   curl http://localhost:3100/ready
   ```

When Loki is ready, the command returns `ready`. Loki can take up to a minute after startup. If you see `Ingester not ready`, wait and run the command again.

Verify that Alloy is collecting logs

1. In a browser, open the Alloy UI at `http://localhost:12345`.

   The home page shows a table of the components in your configuration and their health.

2. Confirm that all four components report **Healthy**.
3. Select **View** on the `loki.source.file.orders` row.

   The component detail page includes debug information for each file the component reads. Confirm that `path` is `/var/log/orders/orders.log` and `is_running` is `true`. The `read_offset` value grows as Alloy reads more of the file.

4. In your terminal, ask Loki which labels it has received:

   ```
   curl -s http://localhost:3100/loki/api/v1/labels
   ```

   The output looks like this:

   ```
   {"status":"success","data":["filename","job","level","service_name"]}
   ```

You set `job` in `local.file_match` and `level` in `loki.process`. The pipeline adds the other two labels for you: `loki.source.file` adds `filename`, and Loki adds `service_name`, which it derives from the `job` label.

Query the logs in Grafana

1. In a browser, open Explore at `http://localhost:3000/explore`.
2. In the data source picker, select **Loki**.
3. Select the **Code** tab to switch the query editor to code mode.
4. Enter a query from the following steps and select **Run query**.

Show all order logs

```
{job="orders-app"}
```

The logs panel adds one line per second.

Show only errors

```
{job="orders-app", level="error"}
```

Because `level` is a label, Loki selects the matching stream directly instead of scanning every line. You see one `payment declined` line for every ten orders.

Filter on a field that isn't a label

```
{job="orders-app"} | json | order_id > 40
```

The `| json` parser extracts every JSON field at query time, so you can filter on `order_id` even though it isn't a label.

Count log lines by level

```
sum by (level) (count_over_time({job="orders-app"}[1m]))
```

This query turns logs into a metric. Grafana shows it as a time series: about 54 `info` lines and 6 `error` lines per minute.

Understand why `level` is a label and `order_id` isn't

Loki indexes labels, not the content of log lines. Each unique combination of label values creates a separate stream.

The `level` field has a small, fixed set of values, so it adds only a few streams and makes filtering by severity fast. In this tutorial it creates exactly two streams, one for `info` and one for `error`.

The `order_id` field is different. Every log line has a different value. If you made `order_id` a label, Loki would create a separate stream for every order. This problem is high cardinality. It increases memory use in Loki and slows down queries.

Keep high-cardinality fields such as IDs, user names, and request paths in the log line, and extract them at query time with a parser such as `| json`, as you did in the third query. Turn a field into a label only when it has a small, bounded set of values and you filter on it often.

> **TIP**
>
> To see how many streams each label creates, run `logcli` with `logcli series --analyze-labels '{job="orders-app"}'`. For this tutorial, the output reports `Total Streams: 2`.

Troubleshoot

If Grafana shows no logs:

- Run `docker compose logs alloy` and look for lines with `level=error`.

- In the Alloy UI, open any component that isn't **Healthy**. The component detail page shows the error message.
- If you edited `config.alloy`, restart Alloy with `docker compose restart alloy`.

If `curl http://localhost:3100/ready` keeps returning `Ingester not ready`, wait about a minute after startup and try again.

If a container fails to start because a port is already in use, change the host port in `docker-compose.yaml`. For example, change `"3000:3000"` to `"3001:3000"` and open Grafana at `http://localhost:3001`.

Clean up

To stop the containers and delete the shared log volume, run:

```
docker compose down -v
```

Next steps

- To send these logs to Grafana Cloud instead of a local Loki, replace the `loki.write` component with one that reads your Grafana Cloud Logs URL, user, and API token from environment variables:

  ```
  loki.write "grafana_cloud" {
    endpoint {
      url = sys.env("GRAFANA_CLOUD_LOKI_URL")

      basic_auth {
        username = sys.env("GRAFANA_CLOUD_LOKI_USER")
        password = sys.env("GRAFANA_CLOUD_API_TOKEN")
      }
    }
  }
  ```

  Remember to update `forward_to` in `loki.process` to `[loki.write.grafana_cloud.receiver]`.
- Read Label best practices to plan labels for your own applications.
- Explore other `loki.process` stages, such as `stage.timestamp`, which uses a timestamp from the log line instead of the time Alloy read it.
- Learn more about LogQL.